

Knowledge Graphs

Memory Cheat Sheet

Layer	Example Functions / Constructs	Purpose
RDF	Alice teaches ML / Triples (subject, predicate, object)	Graph data model for representing knowledge as relationships
Turtle	ex:Alice ex:teaches ex:ML . / Prefixes, blank nodes, literals	Human-readable syntax to serialize RDF
RDFS	rdfs:domain, rdfs:range, rdfs:subClassOf, rdfs:subPropertyOf	Basic schema definition and lightweight inference
OWL	owl:EquivalentClass, owl:Restriction, owl:someValuesFrom, owl:inverseOf, owl:TransitiveProperty	Advanced ontology modeling and logical reasoning
SPARQL	SELECT, ASK, CONSTRUCT, FILTER, OPTIONAL, UNION, GROUP BY	Query and manipulate RDF graphs
SHACL	sh:NodeShape, sh:PropertyShape, sh:minCount, sh:maxCount, sh:datatype, sh:class, sh:pattern , <i>sh:minCount 1 ;</i> <i>sh:maxCount 1 ;</i> <i>sh:path :taxonomyID</i>	Validate RDF graphs against structural and semantic constraints

One-Line Summary to Remember

RDF stores facts,
Turtle writes them,
RDFS adds simple schema,
OWL adds richer logic,
SPARQL queries everything.

RDF Schema (Resource Description Framework Schema, variously abbreviated

as RDFS, RDF(S), RDF-S, or RDF/S) is a set of classes with certain properties using the RDF extensible knowledge representation data model, providing basic elements for the description of ontologies.

Core Idea

A Knowledge Graph represents information as **entities (nodes)** and **relationships (edges)** between them. Instead of storing text and retrieving by similarity, you store *facts* and traverse *explicit connections*.

Vector DB asks: "what text chunks are similar to this query?"

Knowledge Graph: "what is the relationship between these specific entities?"

(Paris) --[capital_of]--> (France)
(France) --[located_in]--> (Europe)
(Eiffel Tower) --[located_in]--> (Paris)
(Eiffel Tower) --[built_by]--> (Gustave Eiffel)
(Gustave Eiffel) --[born_in]--> (Dijon)

ELI5: A vector DB is a library where you find books by how similar their covers look. A knowledge graph is a librarian who knows every book, every author, every relationship — and can answer *"find me all books by authors born in France who influenced Picasso"* in one traversal.

Quick Cheat Sheet

Concept	One-liner
Triple	Subject → Predicate → Object, atomic unit of a KG
Ontology	Schema defining allowed entity types and relations
Multi-hop	Chain multiple relationships to answer complex queries
GraphRAG	Build KG from docs, use graph + vector search together
TransE	Learn entity/relation embeddings: $h + r \approx t$
Neo4j	Most popular graph DB, Cypher query language
Link prediction	Predict missing edges in the graph using embeddings

Interview tip: "Vector DBs retrieve by proximity — they're great for unstructured text but can't reason about explicit relationships. Knowledge graphs store facts as triples and support multi-hop traversal — answering

questions that require chaining several relationships. GraphRAG combines both: use vector search to find the entry point in the graph, then traverse to collect structured context. This is the current best practice for complex enterprise RAG applications where hallucination is costly."

GraphRAG

<https://arxiv.org/pdf/2404.16130>

What is NLP?

Natural Language Processing (NLP) is the broad field of enabling computers to process, understand, and generate human language.

👉 NLP includes many tasks such as:

- Tokenization
- POS tagging
- Parsing
- Sentiment analysis
- Translation
- Summarization
- **NER**

It's my understanding that ontology is the framework, the schema, and knowledge base is once you've added data to the schema and it has meaning.

<http://www.loa-cnr.it/Papers/KBKS95.pdf>

A data lake is a centralized, scalable repository that stores vast amounts of raw, unstructured, semi-structured, and structured data in its native format

Data model vs Ontology (<https://www.youtube.com/watch?v=aQNSt3sjCJM>)

Data model (Structure- Db, passive , application-specific, Close world assumption, project level component)

Ontology : An explicit specification of a conceptualisation , active statement , Open world assumption, formalise domain meaning.

Conceptualisation- Abstract , shared mental model of a domain

Specification (make the model concrete)

Formal (written in a language with well defined logic)

Shared (Represent consensus among a community)

Concept	What It Is	Main Role	Typical Contents
Ontology	A formal conceptual model / schema	Defines concepts, classes, properties, constraints, and relationships	"Species", "Habitat", "producesSound", subclass rules, domain/range constraints
Knowledge Base (KB)	A repository of structured knowledge	Stores facts/ knowledge in machine-readable form	Facts, rules, documents, triples, assertions
Knowledge Graph (KG)	A graph-structured representation of knowledge	Organizes knowledge as entities/nodes + relations/edges	Nodes, edges, semantic relationships, linked entities

conceptualization: an intensional semantic structure which encodes the implicit rules constraining the structure of a piece of reality. Formal Ontology: the systematic, formal, axiomatic development of the logic of all forms and modes of being. ontological commitment: a partial semantic account of the intended conceptualization of a logical theory. ontological engineering: the branch of knowledge engineering which exploits the principles of (formal) Ontology to build ontologies. ontological theory: a set of formulas intended to be always true according to a certain conceptualization

An entity in Natural Language Processing (NLP) is a distinct, identifiable object or concept mentioned in text, such as people, organizations, locations, dates, or product names.

In the knowledge management we search to reduce the gaps, data security, operational experience, we could use

A KMS (knowledge management system) in which we use a

Knowledge based - Which have several types of information like tactic, explicit and implicit knowledge

Una base de conocimientos (*knowledge base* o KB) es un repositorio centralizado y digital que almacena información detallada, útil y organizada sobre productos, servicios, procedimientos o temas específicos de una organización

The idea to manage the knowledge is to have the possibility to create-storage-

sharing

Dif entre knwodlege base and FAQ:

FAQ: preguntas frecuentes

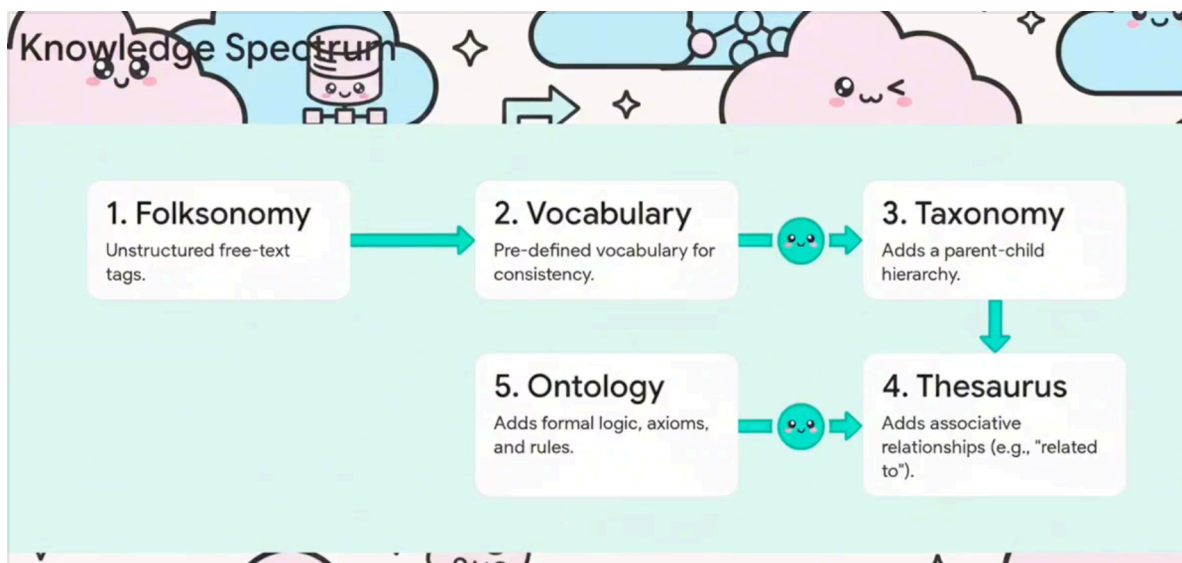
Radica en profundidad de la información, manuales técnicos,

<https://www.youtube.com/watch?v=tBb9FN2sEec>

Concept	Definition	Primary Purpose	Typical Content	Example Use
Content Management System (CMS)	A software platform used to create, organize, edit, publish, and manage digital content.	Manage and publish human-readable content	Web pages, blog posts, media, documentation	Managing a website or internal knowledge portal
Document Management System (DMS)	A system for storing, versioning, indexing, retrieving, and controlling access to documents/files.	Manage document lifecycle and records	PDFs, Word files, reports, scanned documents	Enterprise document repository with version control

Data Warehouse	A centralized repository designed to store integrated, historical, and structured data optimized for querying, analytics, reporting, and machine learning.	Aggregate data for analysis and decision-making	Structured/tabular data from many sources	BI dashboards, trend analysis, ML feature generation
-----------------------	--	---	---	--

Ontology : Knowledge Representations



Vocabulary refers to the knowledge of words as well as the meaning of words

A thesaurus is a reference tool (book or electronic) that lists words grouped by similarity of meaning, specifically providing synonyms and often antonyms. It helps improve vocabulary, prevent word repetition, and increase precision in writing.

Ontology is the branch of metaphysics studying the nature of being, existence, and reality, exploring what entities exist and how they can be grouped or related.

Ontology :

An ontology formally defines a common set of terms that are used to describe and represent a domain. An ontology is domain specific, and it is used to

describe and represent *an area of* knowledge. It contains terms and the relationships among these terms. There is another level of relationship expressed by using a special group of terms: properties. These property terms describe various features and attributes of the concepts, and they can also be used to associate different classes together.

By using **RDFS** and/or **OWL**, you can define an ontology.

<https://blog.thedigitalgroup.com/ontologies-vs-taxonomies-vs-thesauri-and-its-place-on-the-semantic-web>

Automated reasoning

https://drive.google.com/drive/folders/15HNd46z9G2tuN35LzYox8gf_bJbyjNzb

<http://colab.research.google.com/drive/1BJAwSJVFdiJQ5FsHHBm0Q24p1EBW1pus>

The Art of understanding

Meaning is a relationship between two sorts of things:

- ☐ Signs and
- ☐ the kinds of things they intend, express, or signify.
- ☒ Words (and nonverbal symbols) are necessarily meaningful!

Meaning refers to the significance, purpose, intention, or definition behind words, actions, concepts, or signs

Understanding (in general) is the ability to grasp the meaning of information.

- ☒ Information is conveyed in a message using a specific language from a sender to a receiver.

Correct interpretation depends on

Syntax,
Semantics,
Context,
Pragmatics,
Experience

Concept	Focus	Key Question
Syntax	Structure	Is it well-formed?
Semantics	Meaning	What does it mean?
Context	Situation	When/where does it apply?

Pragmatics	Use	How is it used?
Experience	Learning over time	What have we learned?

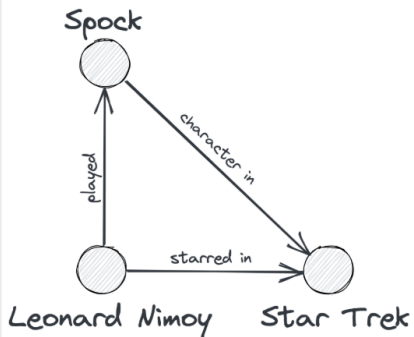
Lexical meaning refers to the specific, dictionary-definition meaning of a word in isolation, independent of its grammatical function, context, or surrounding words

For successful communication,

- information has to be correctly transmitted (Syntax)
- the meaning (Semantics) of the transmitted information must be interpreted correctly (= understanding).
- Understanding depends on
 - the context of both sender and receiver and
 - the pragmatics of the sender.
- (Personal) experience determines how sender and receiver interpret the semantics, context, and pragmatics of a message, and thus its intended meaning.

An Intuitive Way to Represent Knowledge





Definition 1.1:
 A **directed edge-labeled graph** $G=(V,E,L)$ consists of

- a set V of **nodes (vertices)**, $|V|=n$,
- a set L of **edge labels**,
- and a set E of **directed edges**, $E \subseteq V \times L \times V$, where each edge $e_i=(v_k, l_j, v_l)$, $e_i \in E$ is an ordered triple of two vertices $v_k, v_l \in V$ and an edge label $l_j \in L$.

$V=\{\text{Spock, Leonard Nimoy, Star Trek}\}$
 $L=\{\text{played, starred in, character in}\}$
 $E=\{(\text{Leonard Nimoy, played, Spock}), (\text{Leonard Nimoy, starred in, Star Trek}), (\text{Spock, character in, Star Trek})\}$

Knowledge Graphs 2023, Prof. Dr. Harald Sack, FIZ Karlsruhe – Leibniz Institute for Information Infrastructure & Karlsruhe Institute of Technology

6

A knowledge graph is a structured, semantic network representing real-world entities (people, places, concepts) as nodes and their relationships as edges, allowing machines to understand, integrate, and infer data

<https://spacy.io/>

Python library to Industrial strength NLP

Token object

Lemma, part of speech tag,

dependency parsing

tokenization, lemmatization, Part-of-speech (POS) tagging and dependency parsing.

NLP Processing Concepts – Meanings and Examples

Concept	Meaning (Distinct Role)	Key Question	Example Input	Example Output
Tokenization	Splits raw text into basic units (tokens)	How is the text segmented?	"The birds were singing loudly"	["The", "birds", "were", "singing", "loudly"]
Lemmatization	Converts words to their canonical (dictionary) form	What is the base form of each word?	"birds", "singing"	"bird", "sing"
POS Tagging	Assigns grammatical roles to each token	What is each word's grammatical function?	["The", "birds", "were", "singing", "loudly"]	The (Det), birds (Noun), were (Verb), singing (Verb), loudly (Adverb)
Dependency Parsing	Identifies syntactic relationships between words	How are words structurally related?	"The birds were singing loudly"	subject(singing, birds); advmod(singing, loudly)

Lemmatization falls under the umbrella of **morphological analysis**, the study of words. It requires morphological rules and a lexicon to reduce a word to its base form. In this way, words that have different inflections can be treated as the same item. For example, the auxiliary verbs *is*, *are*, *was*, and *were* are grouped together under the lemma *be*.

Part-of-Speech (POS) tagging in NLP is the process of labeling each word in a text with its grammatical role (e.g., noun, verb, adjective) based on its definition

and context.

Parsing : análisis sintáctico

Parsing in Natural Language Processing (NLP), or syntactic analysis, breaks down sentences into component parts to analyze their grammatical structure, producing a parse tree for machine understanding

- **Noun Phrase (Subject):** The man
- **Verb Phrase (Predicate):** opened the door
- **Parts of Speech:** The (article), man (noun), opened (verb), door (noun).

They are two distinct procedures: POS Tagging: each token gets assigned a label which reflects its word class. Parsing: each sentence gets assigned a structure (often a tree) which reflects how its components are related to each other.

Lemmatization and stemming reduce inflected forms to a base: "running" → "run", "geese" → "goose".

Stemming and lemmatization are text normalization techniques in NLP that reduce words to their base forms, though they differ in method and accuracy. Stemming is a fast, rule-based approach that crudely chops off word endings (e.g., "studies"

"studi"). Lemmatization is more accurate, using vocabulary and morphological analysis to return valid, dictionary-meaning words (e.g., "studies" "study").

Stemming vs. Lemmatization Comparison

- **Methodology:** Stemming removes suffixes using crude rules; Lemmatization uses linguistic context and dictionaries.
- **Output Quality:** Stemming often produces non-words ("chang" for "changing"); Lemmatization always produces valid dictionary words.
- **Context Awareness:** Lemmatization understands that "better" becomes "good" (POS tagging); Stemming does not.
- **Speed:** Stemming is generally faster and computationally cheaper; Lemmatization is slower.

Morphological Analysis

Morphological analysis is the process of **breaking words into their smallest meaningful units (morphemes)** and analyzing their structure to understand how meaning is constructed.

What is a Morpheme?

A **morpheme** is the smallest unit of meaning in a language.

- **Root/Base:** core meaning

- "play"
- **Prefix:** added before the root
 - "un-" in "unhappy"
- **Suffix:** added after the root
 - "-ing" in "playing"

Morphological analysis helps:

- Normalize and unify concepts (*dogs* → *dog*)
- Extract richer semantic features (*tense, plurality, repetition*)
- Improve relation extraction

Aspect	Morphological Analysis	Syntactic Analysis
Focus	Internal structure of words	Structure of sentences
Unit	Morphemes	Words & phrases
Example	sing + -ing	subject(sing, birds)
Question	How is the word built?	How is the sentence organized?

What is Chunking?

Definition: is the process of extracting short phrases from a sentence.

Unlike full parsing, it does not specify internal structure of the phrase or its role in the main sentence.

Noun chunking, specifically, aims to identify noun phrases – groups of words built around a head noun that act as a single unit.

Chunking is the process of **grouping tokens into syntactically correlated parts** such as:

- Noun Phrases (NP)
- Verb Phrases (VP)
- Prepositional Phrases (PP)

👉 Concern: *Which words belong together as a unit?*

How to Generate Chunks (Step-by-Step)

1. Tokenization

Split the sentence into words.

"The birds were singing in the forest"

→ ["The", "birds", "were", "singing", "in", "the", "forest"]

2. POS Tagging

Assign grammatical roles.

→ The (Det), birds (Noun), were (Verb), singing (Verb), in (Prep), the (Det), forest (Noun)

3. Apply Chunking Rules (Chunk Grammar)

You define **patterns** based on POS tags.

Examples:

- **Noun Phrase (NP):**
Det + (Adj)* + Noun
- **Verb Phrase (VP):**
Verb + (Adv)*
- **Prepositional Phrase (PP):**
Prep + NP

What is NLP?

Natural Language Processing (NLP) is the broad field of enabling computers to process, understand, and generate human language.

👉 NLP includes many tasks such as:

- Tokenization
- POS tagging
- Parsing
- Sentiment analysis
- Translation
- Summarization
- **NER**

4. Generate Chunks

From the sentence:

Chunk Type	Words
NP	The birds
VP	were singing
PP	in the forest
NP (inside PP)	the forest

Final Chunked Output

[NP The birds] [VP were singing] [PP in [NP the forest]]

Example Table

Step	Output
Tokens	The / birds / were / singing / in / the / forest
POS Tags	Det / Noun / Verb / Verb / Prep / Det / Noun

Chunks	[NP The birds], [VP were singing], [PP in the forest]
--------	--

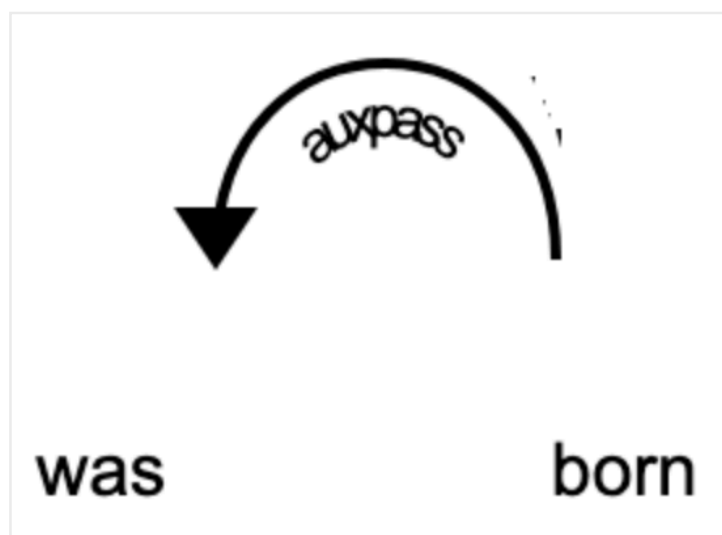
<https://medium.com/@jagadeesan.ganesh/understanding-chunking-algorithms-and-overlapping-techniques-in-natural-language-processing-df7b2c7183b2>
<https://www.geeksforgeeks.org/artificial-intelligence/chunking-strategies/>

Parsing, syntax analysis, or syntactic analysis is a process of analyzing a **string** of **symbols**, either in **natural language**, **computer languages** or **data structures**, conforming to the rules of a **formal grammar** by breaking it into parts. The term *parsing* comes from Latin *pars* (*orationis*), meaning **part (of speech)**.^[1]

dependency parsing

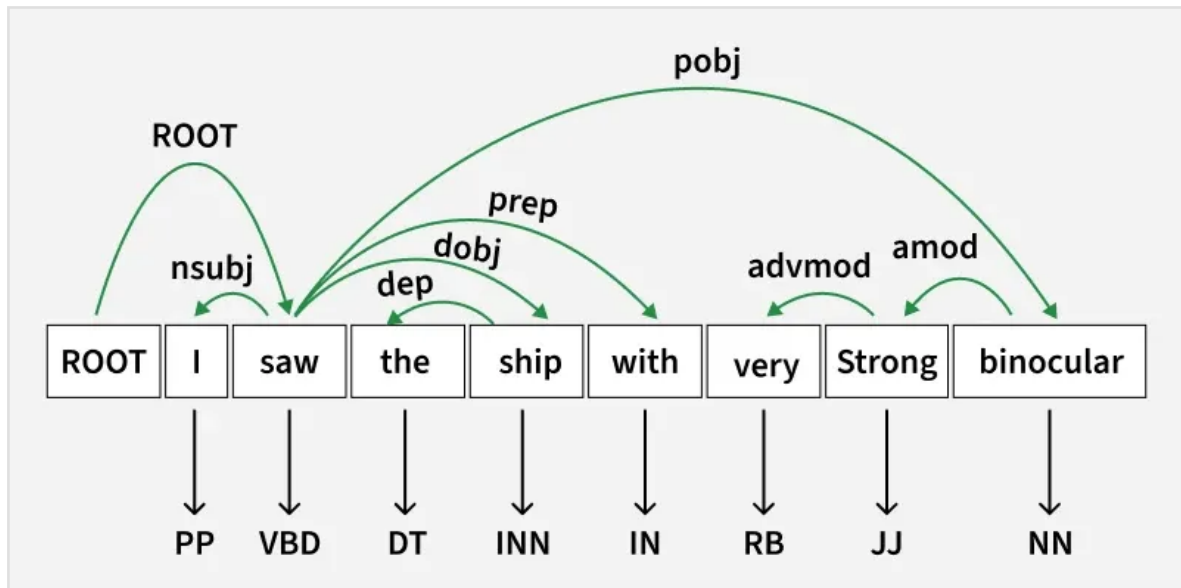
is a process that determines and examines the grammatical relationships between phrases and words in a sentence.

Each relationship is indicated by a **head** (tail of the arrow) and its modifiers, the **dependent/s** (tip of the arrow). In the diagram below, *born* is considered the head, *was* as the tail, and auxpass for passive auxiliary



Dependency parsing in NLP identifies grammatical relationships between words by connecting them to heads in a tree-like structure, defining roles like subject, object, or modifier.

<https://www.geeksforgeeks.org/nlp/dependency-parsing-with-nltk/>



PP: Personal Pronoun ("I")

VBD: Verb ("saw")

DT: Determiner ("the")

INN: Noun, direct object

IN: Preposition ("with")

RB: Adverb ("very")

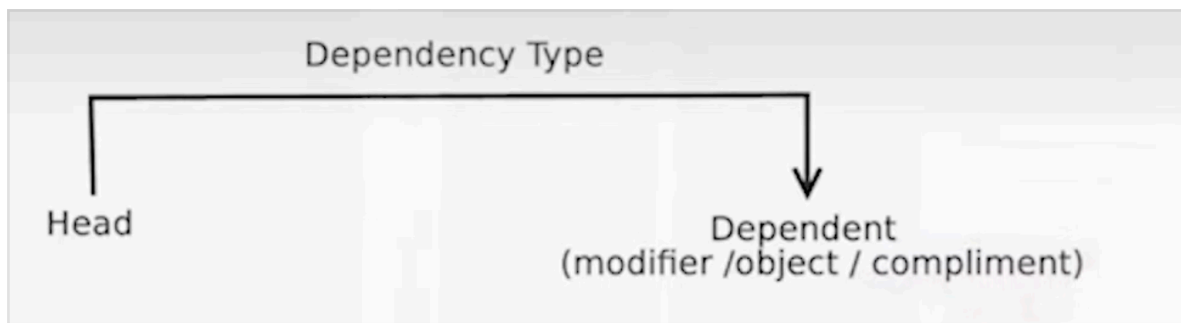
JJ: Adjective ("Strong")

NN: Noun ("binocular")

These terms represent specific dependency labels used in natural language processing (NLP)

- **nsbj (Nominal Subject):** A noun phrase that is the syntactic subject of a clause.
 - Example: [She] runs. (nsbj(runs, She))
- **compound:** A noun that modifies another noun, often part of a compound word.
 - Example: [Wall] Street. (compound(Street, Wall))
- **nsbjpass (Passive Nominal Subject):** A noun phrase that is the syntactic subject of a passive clause.

<https://universaldependencies.org/docsv1/pl/dep/all.html>



D is likely a dependent of head H in construction C :

- H determines syntactic category of C and can often replace C
- H gives semantic specification of C ; D specifies H
- H is obligatory; D may be optional *gray dogs*
- H selects D and determines whether D is obligatory
- The form of D depends on H (agreement or government)
- The linear position of D is specified with reference to H

2.4 Named Entity Recognition (NER)

identify and classify specific entities from unstructured text. These entities can be names of people, organizations, locations, dates, and more.

NER converts raw data into structured information, making it easier for machines to process and understand.

Recall from 1.4 Graphs and Triples that graphs are constructed from *triples*.

Each triple is composed of two vertices connected with a directed edge.

The *vertices* are the entities, while the *edges* represent the relationships that exist between an entity pair.

<https://medium.com/@kanerika/named-entity-recognition-a-comprehensive-guide-to-nlps-key-technology-636a124eaa46>

Named Entity Recognition (NER) acts as a fundamental bridge between unstructured text and structured knowledge bases (KBs), enabling the automatic extraction, classification, and linking of entities. By identifying key information—such as persons, locations, and organizations—NER transforms raw data into actionable knowledge.

Most research on NER/NEE systems has been structured as taking an unannotated block of text, such as **transducing**:
Jim bought 300 shares of Acme Corp. in 2006.
into an annotated block of text that highlights the names of entities:
[Jim]_{Person} bought 300 shares of [Acme Corp.]_{Organization} in [2006]_{Time}.

Lexical Ambiguity

single word or phrase has multiple, distinct meanings,
To solve taxonomical dictionary
Wordnet —NTK library

<https://colab.research.google.com/drive/1NipMe471YhGBg2z-2oAix81NVwCCUXWs#scrollTo=TjCUwG66Cs6x>

Word Sense Disambiguation (WSD)

Disambiguation is the process of resolving ambiguity by identifying the specific meaning of a word, phrase, or entity based on its context
Lesk alg help to find the disambiguation :operates on the premise that words within a given context are likely to share a common meaning.

https://en.wikipedia.org/wiki/Lesk_algorithm

RDF : Resource Description Framework

The Resource Description Framework (RDF) is a W3C standard framework for representing, exchanging, and connecting structured data on the Web. It models information as directed graphs, specifically through "**triples**" (subject-predicate-object URI : Uniform resource identifier) using **Internationalized Resource Identifiers** (IRIs) to identify resources and relationships.

How do I represent the following fact:
"Spock's home planet is Vulcan" in an intuitive way?



URIs Identify and reference resources uniquely

Literals Describe data values that don't have separate existence

Blank Nodes

denote the existence of an individual with specific attributes, but without providing an identification or reference.

Definitions:

○ Let I denote the set of IRIs, L the set of RDF Literals and B the set of RDF blank nodes. The set of RDF terms is defined as $I \cup L \cup B$.

○ An RDF triple $t := (s, p, o)$ is any element of the set $(I \cup B) \times I \times (I \cup B \cup L)$, where $s \in (I \cup B)$ is called the subject, $p \in I$ is called the predicate and $o \in (I \cup B \cup L)$ is called the object.

○ An RDF graph G is a subset of $(I \cup B) \times I \times (I \cup B \cup L)$, i.e. an RDF graph is a set of RDF triples.

Turtle serialisation : <https://zenodo.org/records/10043736>

Definitions:

○ A term t is a word, compound word, or multi-word expression that in specific contexts is given specific meanings.

○ A terminology or vocabulary $V = \{t_1, \dots, t_n\}$ is a set of terms used to describe data in a particular domain or set of domains.

○ A schema is a formal description of the high-level structure of a dataset that may be used for a variety of purposes, including managing, storing, indexing, querying, validating, and/or reasoning over a dataset.

○ A semantic schema is a schema that allows for defining the meaning of high-level terms (aka vocabulary or terminology)

RDF Schema, officially called “RDF Vocabulary Description Language

RDF Classes and properties provide a high-level vocabulary – a set of RDF terms – for general use in RDF descriptions.

- Vocabularies can be easily reused across different independent RDF sources.
- Datasets that agree on vocabularies are better integrable since they “speak the same language”.
- Naming convention:
 - Classes are given upper case singular names (:Occupation, :Person, :FictionalCharacter, ...)
 - Properties are given lower case singular names

RDF Schema allows:

- Definition of classes via `rdfs:Class`
- Class instantiation in RDF via `rdf:type`
- Example:
`:Person rdf:type rdfs:Class .`
`:LeonardNimoy rdf:type :Person .`

RDFS Meta-Classes:

Everything in the RDF model is a resource

Definition of hierarchical relationships:

- Subclasses and superclasses via `rdfs:subClassOf`

RDFS Annotation Properties

to annotate resources with useful (human-readable) information.

- `rdfs:seeAlso` defines a relation of a resource to another, which explains it
- `rdfs:isDefinedBy` subproperty of `rdfs:seeAlso`, defines the relation of a resource to its definition
- `rdfs:comment` comment, usually as text
- `rdfs:label` “readable” name of a resource (contrary to ID)

Complex Data structures

Slides: <https://zenodo.org/records/10043764>

Containers: open

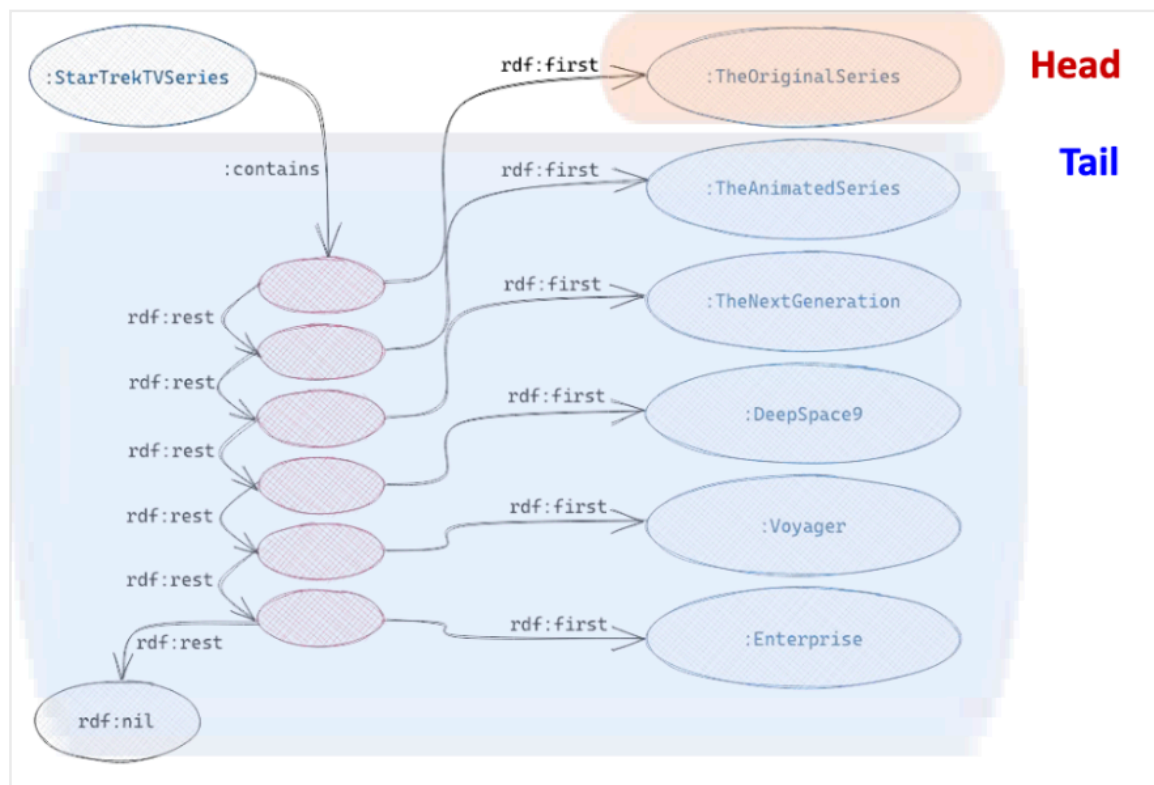
Rdf types

Bag : unordered

Seq: ordered

Alt: Alternatives

Collection: closed list (FIXED)



An RDF dataset is a dictionary of RDF graphs, consisting of:

- one default graph: an RDF graph (that may be empty),
- zero or more named graphs: pairs consisting of
 - i. a name that can be a URI/IRI or a blank

https://en.wikipedia.org/wiki/RDF_Schema

Reification and RDF (Cosificación)

Reification in RDF means:

Turning a statement/triple into a thing you can talk about.

Reification Solves This

You convert the triple into an object/node representing **the statement itself**. Then you can attach metadata to that statement.

Example

Instead of only:

`ex:Alice ex:worksAt ex:OpenAI .`

Reify it:

`ex:stmt1 rdf:subject ex:Alice ;`

```
rdf:predicate ex:worksAt ;  
rdf:object ex:OpenAI ;  
ex:source "LinkedIn" ;  
ex:confidence 0.92 .
```

Reification is representing a triple as an entity so metadata can be attached to that triple.

Reification is the fallacy of treating an abstract concept, idea, or social relation as a concrete, physical thing.

RDF also permits the interleaving of statements,
i.e. to make statements about statements

rdf:Statement
defines an RDF statement consisting of subject, predicate, object

Example:

```
"ChatGPT states that Sherlock Holmes is a role model for Mr. Spock".  
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .  
@prefix : <http://example.org/KG2023#> .  
:statement1 a rdf:Statement ;  
rdf:subject :SherlockHolmes ;  
rdf:predicate :roleModel ;  
rdf:object :Spock .  
:statement1 :statedBy :ChatGPT .
```

- ☐ Modeling data provenance
- ☐ Formalizing statements about reliability and trust
- ☐ Define metadata about statements
- ☒ But you should be careful...
- ☐ Relations and classes can be transformed into instances
potentially resulting in type conflicts
- ☐ Risk to define infinite recursions and cycles

Induction, Deduction, Abduction

The terms infer/inference are often used almost interchangeably with entail/entailment.

- ☐ "entail" refers conceptually to what follows as a consequence,
- ☐ "infer" refers to a process of computing entailments and is very similar to "reason/reasoning".
- ☒ Deductive reasoning involves applying rules over premises to derive

conclusions

and is the main subject of Logic.

● Inductive reasoning involves learning patterns from lots of examples and is the

main subject of Machine Learning.

● Abductive reasoning involves deriving a likely explanation for an observation

Abduction?

Abduction means:

Inferring the best explanation for an observation.

Example:

Observation: Ground is wet

Possible explanation: It rained

And

Yes — **OWL is designed to support deductive inference.**

More precisely:

OWL provides the logical semantics/axioms that allow a reasoner to perform deduction.

OWL → Deductive Inference

Example:

If OWL says:

`ex:Professor rdfs:subClassOf ex:Person .`

And data says:

`ex:Alice rdf:type ex:Professor .`

A reasoner deduces:

`ex:Alice rdf:type ex:Person .`

That is standard OWL reasoning.

In contrast to other data definition languages, RDF(S) is based on formal semantics.

● Formal semantics enables RDF(S) to draw valid and sound logical inferences.

Domain

The **domain** of a property specifies:

What type of subject is expected to use this property.

Example

If we define:

```
ex:teaches rdfs:domain ex:Professor
```

It means:

Whenever something uses ex:teaches as a subject, it can be inferred to be a Professor.

Range

The **range** of a property specifies:

What type of object is expected for this property.

Example

```
ex:teaches rdfs:range ex:Course
```

RDFLib - RDF Serialisation and Visualisation

<https://colab.research.google.com/drive/1qn41H0huz0R1mmUFKD4NOAUW0X9HHxX7>

RDF Graph Manipulation

https://colab.research.google.com/drive/1ppbr3yE_cOPuqc_Ubjap3mt5xHvbnSO

Complex query

<https://query.wikidata.org/>

```
PREFIX wd: <http://www.wikidata.org/entity/>
PREFIX wdt: <http://www.wikidata.org/prop/direct/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
SELECT ?authorLabel ?bookLabel ?date
WHERE {
  ?book wdt:P31 wd:Q47461344 .
  ?book wdt:P50 ?author .
  ?book wdt:P577 ?date .
  ?book rdfs:label ?bookLabel
  FILTER (LANG(?bookLabel)="en")
  FILTER REGEX (?bookLabel,"mars$", "i") .
  ?author rdfs:label ?authorLabel
  FILTER (LANG(?authorLabel)="en") .
} ORDER BY ?date
```

Knowledge Representation Stack:

OWL / RDFS ← semantics / ontology layer
RDF ← graph data model
Turtle ← serialization syntax
SPARQL ← query language

Mini Knowledge Graph Example

We want to represent:

"Alice teaches Machine Learning."

1. RDF (Data Model)

Conceptually, RDF stores the triple:

Alice — teaches — MachineLearning

That is the abstract graph statement.

2. Turtle (Serialization Syntax)

How we write that RDF triple in text:

```
@prefix ex: <http://example.org/> .
```

```
ex:Alice ex:teaches ex:MachineLearning .
```

Same RDF graph, just serialized in Turtle.

3. RDFS (Basic Schema / Typing)

Add domain/range information:

```
ex:teaches rdfs:domain ex:Professor ;  
           rdfs:range ex:Course .
```

Meaning:

- whoever teaches is a Professor
- whatever is taught is a Course

From this, a reasoner infers:

```
ex:Alice rdf:type ex:Professor .  
ex:MachineLearning rdf:type ex:Course .
```

4. OWL (Richer Ontology Logic)

Add stronger semantic knowledge:

```
ex:Professor owl:equivalentClass ex:FacultyMember .
```

Meaning:

Professor and FacultyMember are logically the same class.

So if Alice is a Professor, infer:

```
ex:Alice rdf:type ex:FacultyMember .
```

5. SPARQL (Query Language)

Query the graph:

```
SELECT ?x
WHERE {
  ?x rdf:type ex:FacultyMember .
}
```

Result:

Alice

Example

```
@prefix ex: <http://example.org/> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix sh: <http://www.w3.org/ns/shacl#> .
```

```
#####
# 1. RDFS (SCHEMA / BASIC VOCABULARY)
#####
```

```
ex:Dog a rdfs:Class .
ex:Person a rdfs:Class .
```

```
ex:hasOwner a rdf:Property ;
  rdfs:domain ex:Dog ;
  rdfs:range ex:Person .
```

```
#####
# 2. OWL (LOGICAL SEMANTICS / INFERENCE RULES)
#####
```

```
ex:PetDog a owl:Class ;
  owl:equivalentClass [
```



```

a owl:Class ;
owl:intersectionOf (
  ex:Dog
  [ a owl:Restriction ;
    owl:onProperty ex:hasOwner ;
    owl:someValuesFrom ex:Person ]
)
] .

#####
# 3. SHACL (VALIDATION RULES / CONSTRAINTS)
#####

ex:DogShape a sh:NodeShape ;
  sh:targetClass ex:Dog ;

  sh:property [
    sh:path ex:hasOwner ;
    sh:minCount 1 ;
    sh:maxCount 1 ;
    sh:class ex:Person ;
  ] .

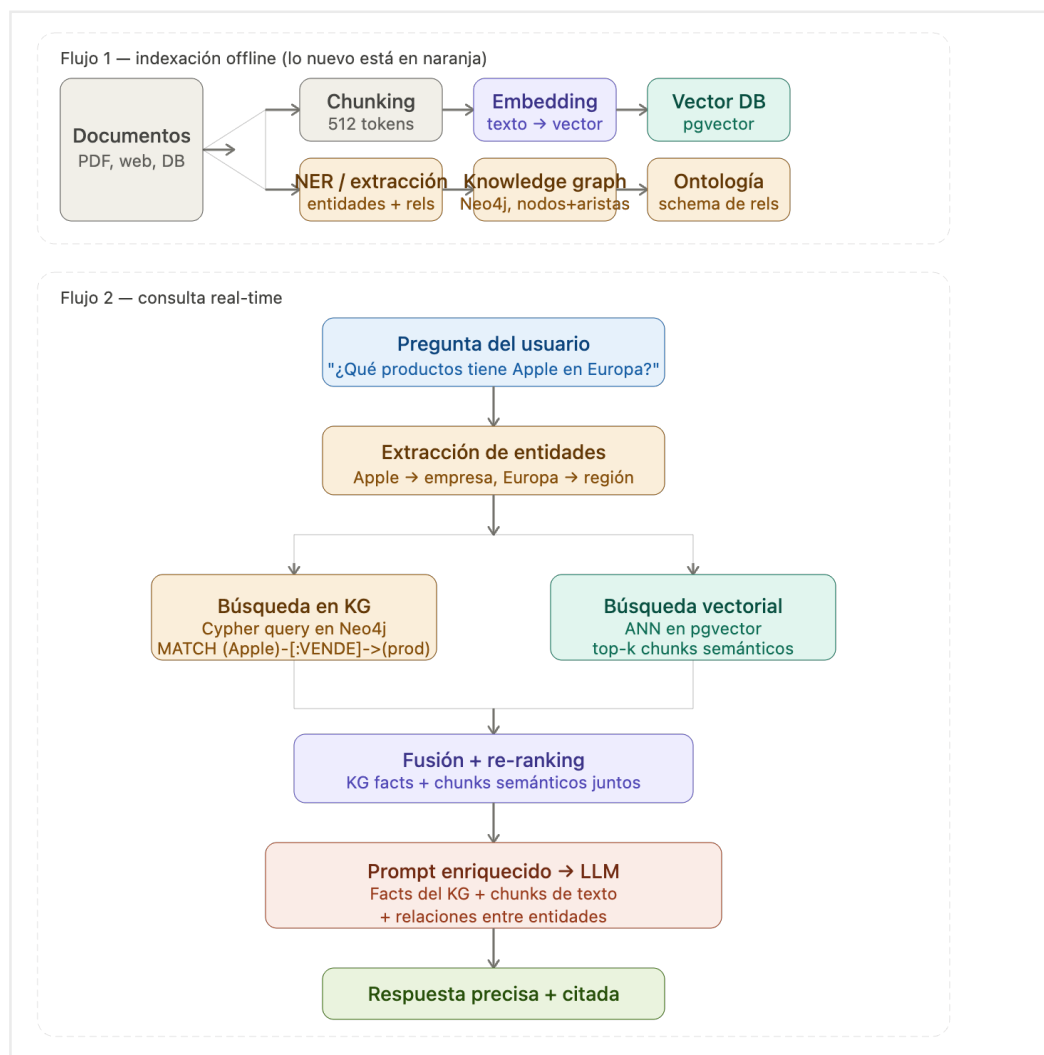
#####
# 4. DATA (KNOWLEDGE GRAPH INSTANCES)
#####

ex:Fido a ex:Dog ;
  ex:hasOwner ex:Alice .

ex:Alice a ex:Person .

```

Si quieres...	Mejor recurso
Base general para grounding de entidades	Wikidata
Taxonomías limpias / jerarquías	YAGO / WordNet
Relaciones semánticas de lenguaje natural	ConceptNet / BabelNet
Geografía / lugares	GeoNames
Reusar ontologías existentes	LOV
Ontología formal de alto nivel	OpenCyc / SUMO



Lo que cambia vs RAG clásico son dos cosas:

En la **indexación** agregas una segunda rama — además de chunking y embedding, extraes entidades y relaciones de los documentos con NER (Named Entity Recognition), y las guardas en un grafo tipo Neo4j con un schema definido por una ontología. Por ejemplo: (Apple) -[:VENDE]-> (iPhone) -[:DISPONIBLE_EN]-> (Europa).

En la **consulta** el flujo se bifurca — la pregunta se analiza para detectar entidades, luego lanzas dos búsquedas en paralelo: una Cypher query en el KG que sigue relaciones entre entidades, y la búsqueda vectorial clásica. Fusionas ambos resultados y construyes un prompt que tiene tanto hechos estructurados del grafo como texto semántico de los chunks.

RAG clásico busca por similitud semántica — encuentra texto parecido a la pregunta. KG-RAG además entiende relaciones entre entidades — puede responder "¿qué productos de Apple se venden en Europa?" siguiendo la cadena Apple → producto → región, algo que la búsqueda vectorial no puede hacer bien porque es una pregunta relacional, no semántica.

Cuándo usarlo: cuando tu dominio tiene muchas relaciones entre entidades

(productos, personas, organizaciones, normativas). No lo usas para documentación técnica simple — ahí el RAG clásico es suficiente y mucho más barato de mantener.

RDF + RDFS + SPARQL es el stack semántico del W3C. Todo se representa como tripletas sujeto → predicado → objeto. RDFS define el schema formal (clases, subclases, propiedades), OWL añade lógica de inferencia, y SPARQL es el lenguaje de query. Se guarda en un **triplestore** como Apache Jena o GraphDB. Es un estándar abierto — cualquier sistema que hable RDF puede consumir tus datos. La desventaja es que es verboso y más lento para queries complejas.

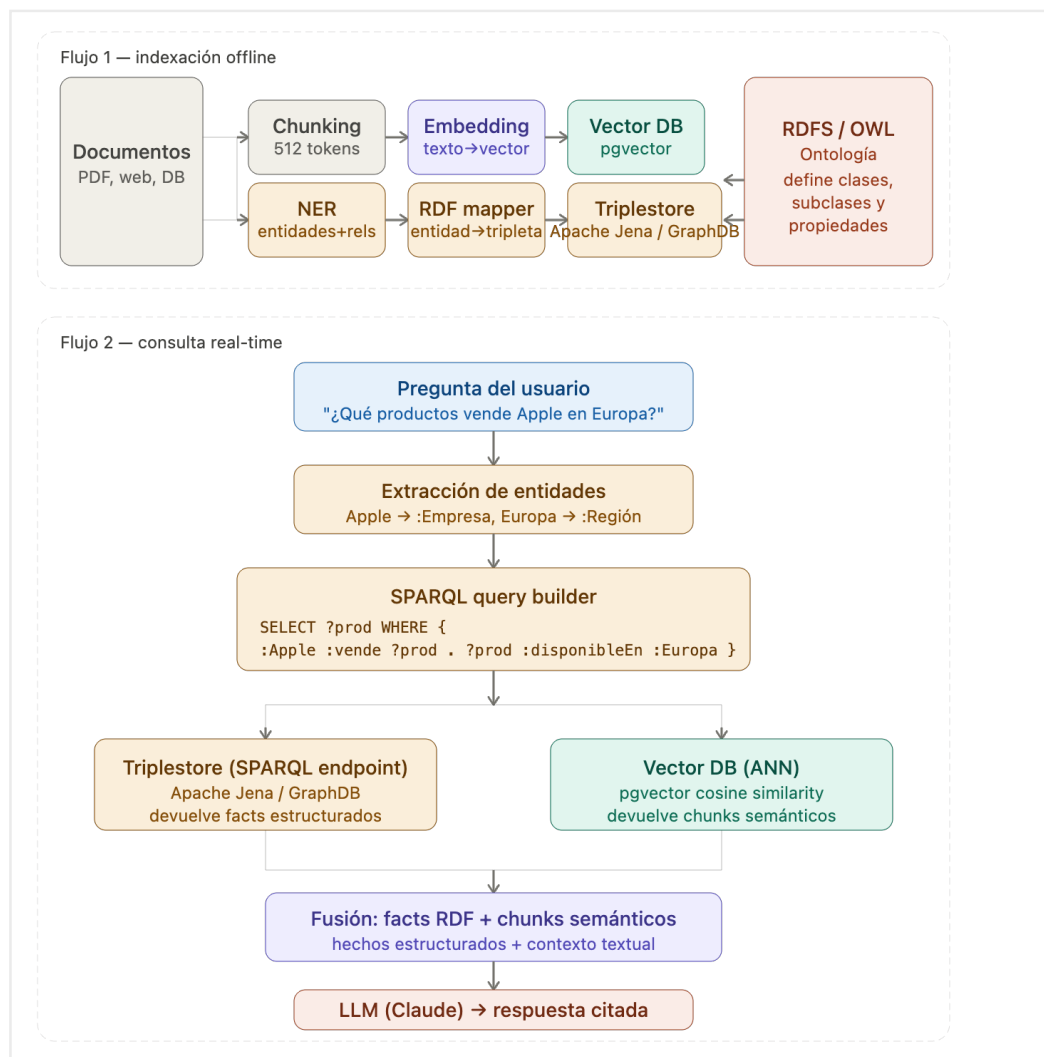
Neo4j

Neo4j usa el modelo **Property Graph** — nodos y aristas que pueden tener propiedades directamente (como {name: "iPhone", precio: 999}). Cypher es su lenguaje de query, mucho más intuitivo visualmente. Es más rápido, más fácil de desarrollar, pero es propietario y no interoperable de forma nativa.

RDF/SPARQL/RDFS

lo usas cuando necesitas **interoperabilidad** entre sistemas, cumplir estándares (gobierno, salud, linked data), o hacer **inferencia lógica** automática — por ejemplo, si iPhone es subclase de Producto y Producto tiene precio, el sistema infiere que iPhone tiene precio sin que lo definas explícitamente.

Neo4j lo usas cuando priorizas **velocidad de desarrollo y performance** — startups, productos, sistemas internos donde no necesitas que otros consuman tu grafo con estándares abiertos.



¿Cómo se llama la "base de datos" de SPARQL?

Se llama **Triple Store** (o *RDF Store*). Es el motor de almacenamiento nativo para RDF.

Tu "base de datos" SPARQL = Triple Store

- └─ almacena triples: (sujeto, predicado, objeto)
- └─ ejemplos: GraphDB, Apache Jena Fuseki, Stardog, Amazon Neptune, Virtuoso

Los términos se relacionan así:

Knowledge Graph

- |
- | └─ es el CONCEPTO (la red de entidades y relaciones con semántica)
- |
- | └─ vive DENTRO de un Triple Store (RDF) o Graph DB (LPG)
 - | └─ se consulta con SPARQL (si es RDF) o Cypher (si es Neo4j/LPG)

Knowledge Base es el término más amplio — incluye las ontologías, reglas,

vocabularios controlados, y los datos. El Knowledge Graph es la instancia concreta de datos. El Triple Store es donde físicamente se guarda.

OWL (Web Ontology Language) es el lenguaje con el que le dices al knowledge graph qué *significa* cada cosa — no solo cómo se llama, sino qué implica. Es la diferencia entre guardar datos y guardar conocimiento.

La idea central: en RDF puro dices hechos. En OWL, dices **reglas** — y el razonador *deduce* hechos nuevos automáticamente.